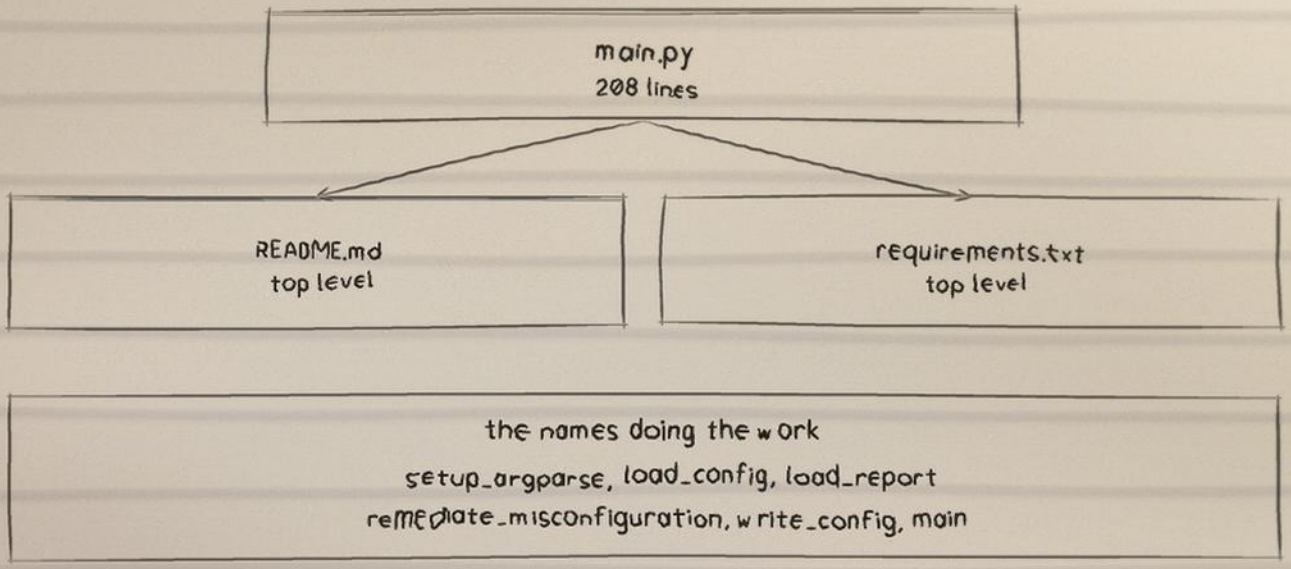
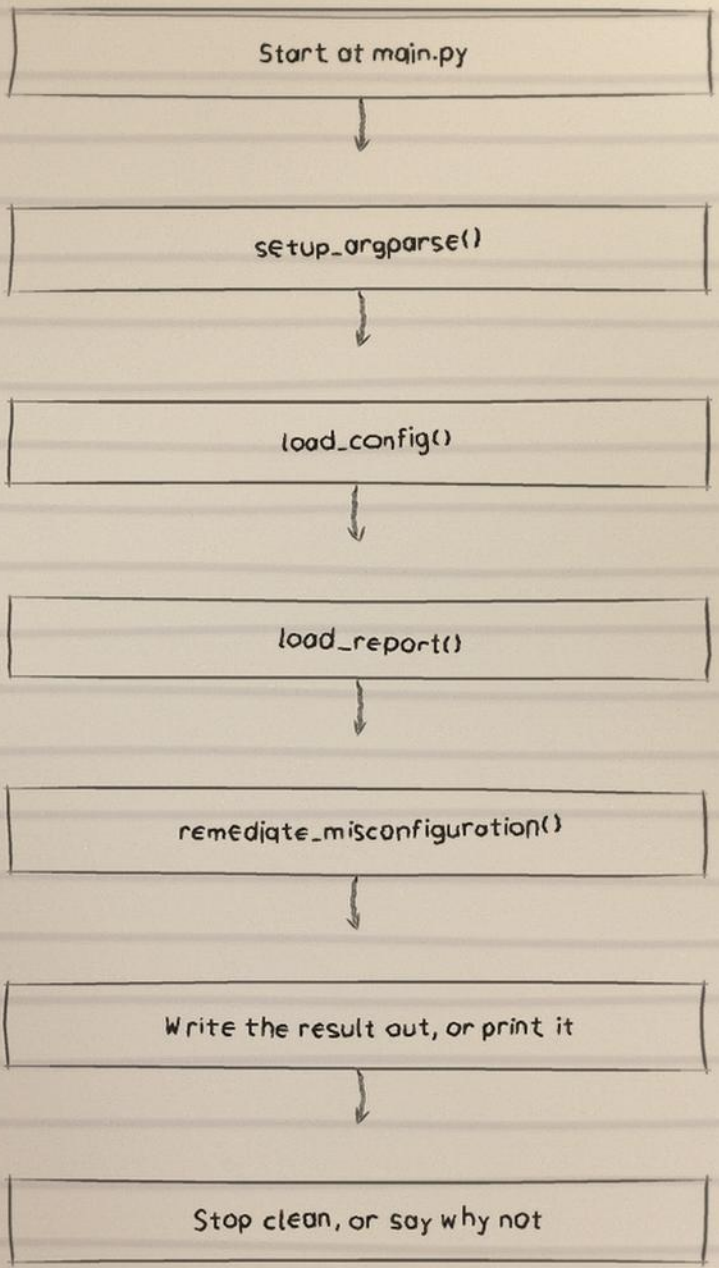
Misconfig Auto Remediator: how it is put together



sketched from the files

Misconfig Auto Remediator: the run, step by step



If a step fails, the honest run says so and stops.

traced, not guessed GK

Product map: what Misconfig Auto Remediator is for

WHAT IT IS

A small project in Python, Markdown, Text. The README and the files are the whole claim.

WHAT GOES IN

The files at the top of the tree: main.py, README.md, requirements.txt.

WHAT COMES OUT

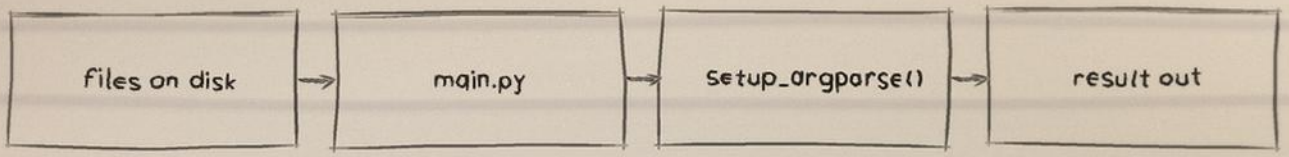
What the top files produce when they run or get read. Nothing else is promised.

WHERE IT STOPS

No service behind it, no account, no upload. If a path is untested, VISUAL.md says so.

mapped by hand, GK

Misconfig Auto Remediator: data flow, in one direction

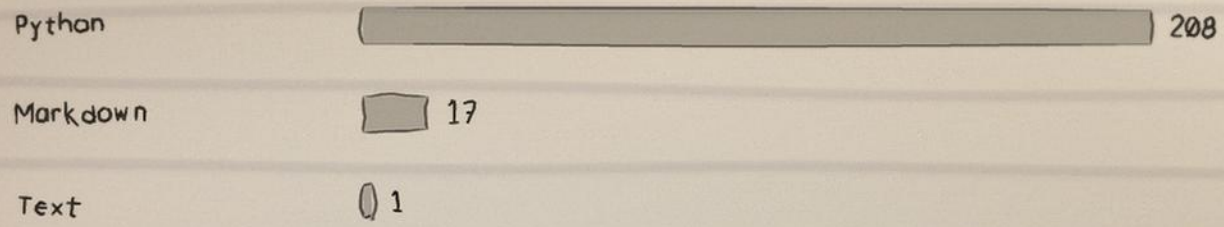


One direction. Anything the code does not do is not drawn.

second sheet, GK

Misconfig Auto Remediator in numbers, counted 2026-10-30

Counted from the local snapshot of this repo. Method for every number: VISUAL.md.



3 files in the tree.

4 commits, first 2025-03-30, last 2025-03-30, from the local git log.

No test files by name in this snapshot.

counted and drawn, GK

Misconfig Auto Remediator: schematic of the apparatus

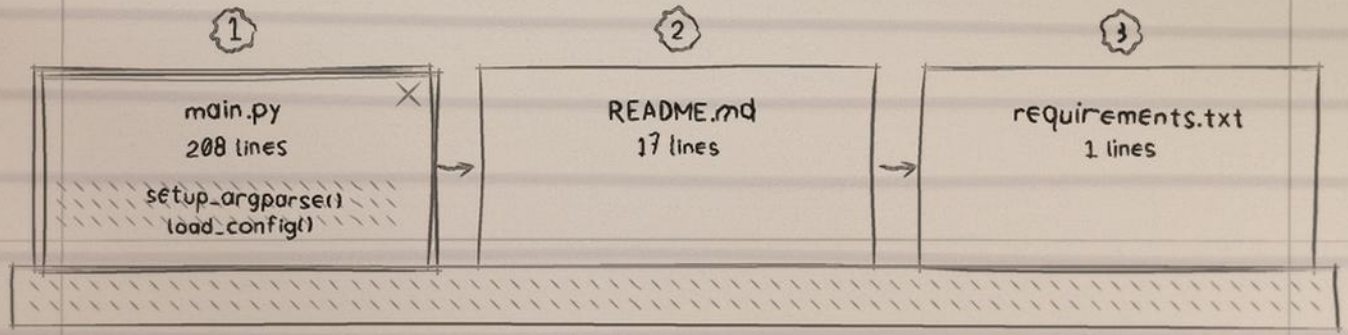
the system as a machine: parts, flow, and measurements

construction lines left in, as drawn

parts list

1. main.py
2. README.md
3. requirements.txt

in flow order, first to last



base plate: the repo itself, sectioned

span: 3 files, 226 lines counted

turns of the crank so far: 4 commits, 2025-03-30 to 2025-03-30

measured and sketched, GK